MapReduce with Direct Data Access on Shared Block Storage

Technical Research Report

WooJun Lim

wjlim.eric@gmail.com

1. Project Overview

This work explores a MapReduce data-access model in which intermediate data is accessed directly over shared block storage rather than transferred through application-level services.

Conventional MapReduce implementations exchange intermediate data between workers through explicit transfer stages mediated by components such as shuffle handlers and fetchers. These components operate as long-running services that coordinate cross-node intermediate data movement.

Under the direct-access model, reducers read intermediate files directly from shared iSCSI block storage. As a result, intermediate data is exchanged through the storage layer, eliminating the need for dedicated transfer services.

To avoid introducing additional filesystem-level coordination, the model operates without a shared filesystem layer. This work therefore focuses on understanding the constraints that arise when intermediate files are accessed directly from shared block storage in a multi-node environment and on examining how such constraints can be addressed.

2. Data-Access Model

In MapReduce, intermediate data exchange follows a constrained access pattern. Each intermediate file is written once by a mapper and subsequently read by reducers, forming a single-writer, multiple-reader access pattern that largely removes the need for concurrent write coordination and write–read coordination.

The direct-access model combines this access pattern with shared block storage, which is exported to worker nodes through iSCSI volumes. Instead of relying on application-level transfer services, workers access intermediate files directly from shared storage without a

shared filesystem layer for coordination.

Because this access takes place across nodes without filesystem-level metadata coordination, maintaining consistent file visibility becomes a potential system constraint in the direct-access model.

3. System Constraints and Analysis

When shared iSCSI block storage is accessed by multiple nodes without a shared filesystem layer, filesystem metadata is cached independently on each node, which can lead to inconsistencies in file visibility across nodes.

To examine how these inconsistencies arise in practice, a series of experiments were conducted to observe the behavior of file creation, lookup, and access across worker nodes. The investigation focused on identifying the conditions under which reducers fail to locate intermediate files written by mappers and analyzing the kernel-level mechanisms involved in these failures.

3.1. File Visibility Failures in Shared Block Storage Access

In the initial experiment, a mapper created an intermediate file and wrote data using the `write()` system call. Since `write()` updates only the page cache and marks the corresponding pages as dirty, additional steps were required to ensure that the data reached the shared disk.

The mapper therefore explicitly flushed cached data using `fsync()` on the file, followed by `fsync()` on the corresponding block device. The first call forced dirty pages to disk, while the second ensured that metadata blocks, such as directory blocks, were written to the shared volume.

After these flush operations, reducers were able to open and read the file successfully. Because their caches were initially empty, the file data and metadata were fetched directly from the shared volume.

However, when the mapper subsequently created another file and repeated the same write and flush sequence, reducers failed to open the newly created file. Although the file had already been written to the shared volume, reducers continued to operate with cached directory metadata reflecting the earlier state of the directory.

3.2. Kernel Metadata Cache Analysis

Closer examination revealed that these failures were related to kernel-level filesystem caching behavior. In Linux filesystems, directory entries and directory blocks may remain cached locally on each node. When multiple hosts access the same shared block storage device

without a shared filesystem layer providing coordination, these cached structures can become inconsistent across nodes.

When reducers attempted to open the newly created file, the kernel's pathname lookup process consulted the local dentry cache. Because the reducer's cached directory block still contained the earlier entries and did not include the newly created file entry, the lookup failed to locate the file entry. As part of the normal lookup process, the kernel created a negative dentry representing the absence of the file and cached it locally on the reducer node. Because this negative entry persisted in the lookup path, subsequent `open()` attempts continued to fail.

To recover from the `open()` failures, cache invalidation was attempted on the reducer nodes. In particular, the dentry cache was cleared using the `drop_caches` interface, and the buffer cache of the block device was flushed using `ioctl(BLKFLSBUF)`.

However, even after these cache invalidation attempts, reducers were still unable to open the newly created file. Further analysis showed that the negative dentry created during the initial failed lookup remained in the dentry cache. Because the `drop_caches` operation does not reliably remove recently referenced entries, the negative dentry persisted in the lookup path, causing repeated `open()` failures.

3.3. Constraint Summary

The observations above reveal a key constraint of the direct-access model when shared iSCSI block storage is accessed by multiple nodes without a shared filesystem layer.

In this configuration, filesystem metadata is maintained in node-local kernel caches. Pathname resolution therefore depends on the state of these caches rather than on a coordinated filesystem metadata view. Consequently, updates made by one worker may not become immediately visible to other nodes.

This behavior highlights the need for engineering approaches that maintain consistent file visibility across nodes.

4. Engineering Approach

As noted in the previous analysis, once a negative dentry is created during a failed lookup and cached in the dentry cache, subsequent cache invalidation does not reliably restore access to the file.

4.1. Cache Invalidation Attempt

A possible workaround was to invalidate filesystem caches before each `open()` operation so that pathname resolution occurred against a fresh cache state, preventing it from consulting

stale directory metadata. Under this approach, reducers consulted the latest on-disk directory metadata and successfully located newly created files.

In practice, however, this approach was not suitable for repeated execution. Cache invalidation mechanisms do not allow selective eviction of metadata associated with a specific file or directory. Instead, they invalidate broad portions of the filesystem cache. Performing such operations before every `open()` would introduce substantial overhead and interfere with normal caching behavior during task execution.

4.2. Pre-allocated File Approach

Rather than repeatedly invalidating caches, the design shifted toward maintaining a stable directory state across nodes even when metadata was cached locally.

This led to a pre-allocated file approach in which the predefined set of files on the shared storage was created in advance and reused throughout execution as intermediate data containers.

Because all workers observed the same directory structure from the beginning, directory lookups operated on a stable set of pre-existing file entries. As a result, reducers were able to open the intermediate files without encountering lookup failures caused by stale directory metadata.

4.2.1 File Initialization

An initial attempt to create the pre-allocated files used `fallocate()`. Although `fallocate()` reserves disk space, the allocated blocks remain uninitialized. Because the pre-allocated approach allows each worker to cache filesystem metadata locally, the state of these uninitialized blocks can also be cached. In such cases, subsequent `read()` operations may return zero-filled data rather than the intended contents.

To avoid this behavior, the files were explicitly initialized through sequential writes, for example using the `dd` utility. This ensured that the underlying blocks were both allocated and initialized before the files were used to store intermediate data.

4.3. Access Procedure

With the pre-allocated file approach, the direct-access model operates as follows.

1. **File pre-allocation**
   Before the shared iSCSI storage is mounted on worker nodes, a set of pre-allocated files is created on the shared storage device.

2. **Mapper writes**
   Each mapper writes its intermediate output to its assigned pre-allocated file. After writing, the mapper invokes `fsync()` on the file to make the written data durable on the shared storage.

3. **Reducer reads**
   Reducers read intermediate data using `O_DIRECT`, bypassing the page cache and accessing the shared storage device directly.

5. Prototype Implementation

To examine the feasibility of the direct-access model, a prototype was implemented and tested on clusters connected through shared iSCSI storage.

5.1. Prototype Topology

Each worker node exposes a local iSCSI target for storing intermediate outputs while simultaneously mounting the targets of peer nodes. Under this configuration, intermediate data produced by mappers remain on the shared storage volume of the mapper node, while reducers access those files through remote mounts of the same volumes.

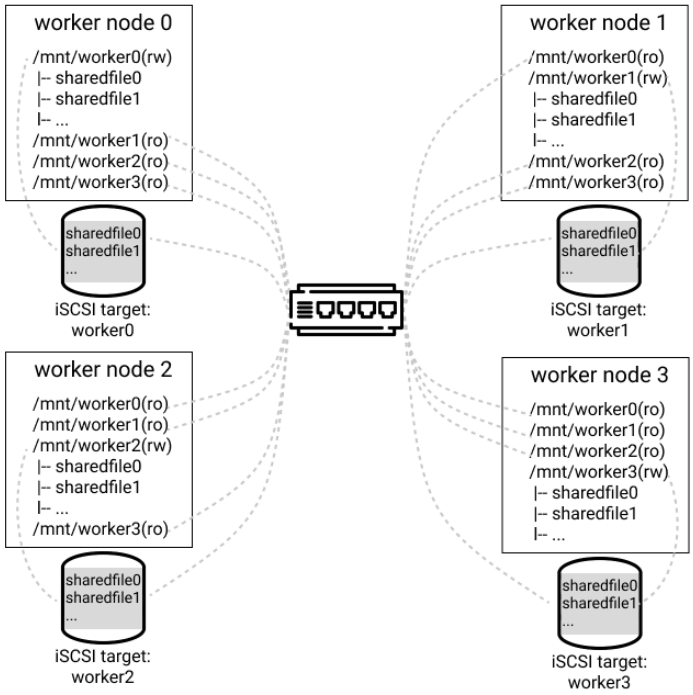


Figure 1: Storage topology of the prototype cluster with local iSCSI targets and remote mounts for intermediate data access.

Each node therefore consists of the following components:

- A **local iSCSI target** mounted read-write mode for intermediate outputs.
- A set of **pre-allocated file**s within the local target volume.
- **Remote iSCSI targets** from peer nodes mounted in read-only mode for reducer access.

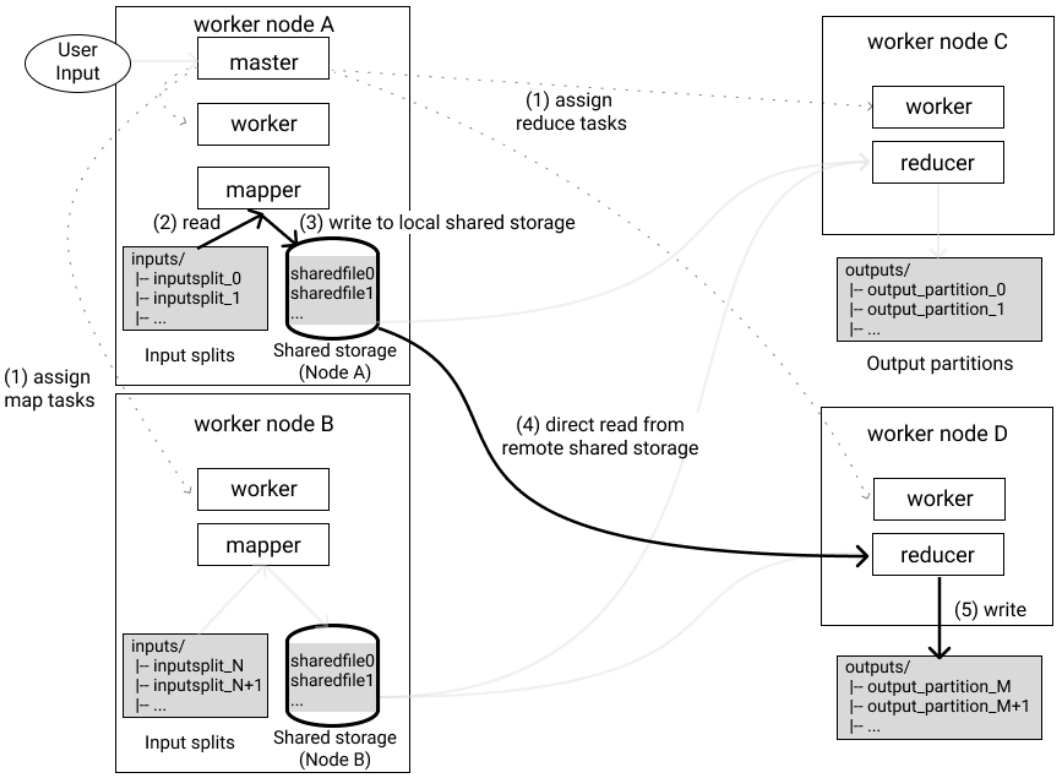


Figure 2: Execution flow of the direct-access MapReduce prototype. Mappers write intermediate data to node-local shared volumes, and reducers read the files directly from the remote shared volumes.

5.2. Execution Flow

The execution of the prototype follows the standard MapReduce control flow while relying on shared block storage for intermediate data access, as described in **Section 4.3**.

1. The master process assigns map and reduce tasks to worker nodes.
2. Each mapper reads its assigned input split and executes the user-defined **map** function.
3. Mappers write intermediate output to their assigned **pre-allocated files** located on the mapper node's shared storage volume.
4. After reducers are notified by the master about the locations of intermediate files, they open the corresponding files through the mounted iSCSI volumes of the mapper nodes and read the intermediate data directly from the shared storage.
5. Once all intermediate data have been processed, reducers execute the user-defined **reduce** function and write the final output partitions to their local disks.

5.3. Test Environment

The prototype was tested on clusters consisting of 4-node and 9-node configurations connected over a 1 Gbps LAN. The iSCSI targets were configured using `targetcli`, and the initiators

used `open-iscsi`.

Three environments were used for testing, summarized in Table 1.

Table 1: Test Environments

Environment	CPU	RAM	Storage	OS	Kernel
Local VM (×4)	Intel i7-10700K	16 GB	64 GB SSD	CentOS 7	3.10.0
AWS t3.medium (×4)	Intel E5-2686 v5	4 GB	16 GB EBS HDD	CentOS 7	3.10.0
PC (×9)	Intel i7-6500U	8 GB	1 TB HDD	CentOS 7	3.10.0

5.4. Execution Results

Across all testing environments, reducers were able to open and read intermediate files stored on mapper-node shared storage through iSCSI volumes without relying on a distributed filesystem layer.

These results indicate that the direct-access model operates correctly under the MapReduce intermediate data access pattern.

The full implementation source code and supporting scripts are available at:
`https://github.com/wjnlim/direct-access-mapreduce`

6. Discussion

The analysis in the preceding sections shows that the main challenge in the direct-access model lies not in block-level data sharing itself but in the visibility of filesystem metadata across nodes. Although intermediate data reside on shared block storage, pathname resolution on each node relies on locally cached filesystem metadata. Consequently, access to intermediate files depends on the state of node-local metadata caches rather than on the availability of the underlying data blocks.

This observation suggests that, in the absence of a shared filesystem layer, maintaining metadata visibility becomes the primary coordination issue for cross-node intermediate data access.

The access semantics of MapReduce intermediate data, combined with a shared storage structure and explicit storage-level I/O control, allow reducers to access intermediate data directly through shared storage.

Taken together, the analysis and prototype execution indicate that part of the coordination typically provided by a shared filesystem layer can instead be handled by the direct-access model. As a result, workers can access intermediate data directly from shared storage without relying on application-level transfer services.

Background References

Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. In *Sixth Symposium on Operating System Design and Implementation*, pages 137-150, San Francisco, CA, 2004.

Linux kernel source code (Version 3.10.0) [Source code].

D. Bovet and Marco Cesati. Understanding the Linux kernel (3rd Edition). O'Reilly, 2005.

Apache Software Foundation. (2018). Apache Hadoop 2.9.2 documentation. `https://hadoop.apache.org/docs/r2.9.2/`.

Tom White. Hadoop: The Definitive Guide (4h Edition). O'Reilly, 2015.

Wickham, H. The Split-Apply-Combine Strategy for Data Analysis. In *Journal of Statistical Software 40 (1)*, pages 1-29, 2011.